《计算机程序的构造和解释》期末考试（A 卷 参考答案和评分标准）

（2023-2024 秋季学期，授课教师：冯新宇、李樾，考试方式：闭卷）

学号：_____ 姓名：_____ 院系：_____ 得分：____（满分 100）

题目	1	2	3	4	5	6	7
得分							

注意事项（请仔细阅读）：

1. 试题的难度不是线性递增的，**注意时间分配**。在这项测验中，可能有些题目较难，因此你不要在一道题上思考时间太久，遇到不会答的题目可先跳过去，如果有时间再去思考。否则，你可能没有时间完成后面的题目。
2. 代码填空题的**正确答案不止一种**，答案正确即可得分。每空**最多只能填一个** Python、Scheme、SQL 的语句或表达式。可以直接使用的 Python 内置函数包括：`all`，`any`，`bool`，`dict`，`enumerate`，`filter`，`float`，`int`，`iter`，`len`，`list`，`map`，`max`，`min`，`pow`，`print`，`range`，`reversed`，`round`，`sorted`，`sum`，`zip`。
3. 可能会用到的链表和树两种抽象数据结构的 Python 实现如下：

```
class Link:
    empty = ()

    def __init__(self, first, rest=empty):
        self.first = first
        self.rest = rest

    def __repr__(self):
        if self.rest is not Link.empty:
            rest_repr = ', ' + repr(self.rest)
        else:
            rest_repr = ''
        return 'Link(' + repr(self.first) + rest_repr + ')'

    def __str__(self):
        string = '<'
        while self.rest is not Link.empty:
            string += str(self.first) + ' '
            self = self.rest
        return string + str(self.first) + '>'

class Tree:
    def __init__(self, label, branches=[]):
        self.label = label
        self.branches = list(branches)

    def is_leaf(self):
        return not self.branches

    def __repr__(self):
        if self.branches:
            branch_str = ', ' + repr(self.branches)
        else:
            branch_str = ''
        return 'Tree({0}{1})'.format(self.label, branch_str)

    def __str__(self):
        def print_tree(t, indent=0):
            tree_str = '  ' * indent + str(t.label) + "\n"
            for b in t.branches:
                tree_str += print_tree(b, indent + 1)
            return tree_str
        return print_tree(self).rstrip()
```

1 Object Oriented Programming (25pts)

Tom think that strings should be registered (备案) before they are printed. Here's his `String` class that does this job. Help him complete the code: (2pts)

```
class String:
    def __init__(self, value):
        self.value = value
        self.registered = False

    def __str__(self):
        if self.registered:
            return self.value
        else:
            return f"Not␣registered␣string"

    def __repr__(self):
        return self.__str__()

    def __add__(self, other):
        """ same behavior as normal str's __add__,
            but returns a String instead of str """
        # type(other) == String
        return String(self.value+other.value) (1分)

    def __mul__(self, other):
        """ same behavior as normal str's __mul__,
            but returns a String instead of str """
        # type(other) == int
        return String(self.value*other) (1分)

    def register(self):
        self.registered = True
```

What would Python display? Write your answer in each blank line. (6pts)

```
>>> a = String("love␣SICP")
>>> print(a)
Not registered string (1分)
>>> a.register()
>>> print(a)
love SICP (1分)
>>> print(a * 2)
Not registered string (1分)
```

```
>>> (a * 2).register()
>>> print(a * 2)
Not registered string (2分)
>>> print("love␣SICP")
love SICP (1分)
```

Now Tom wants to introduce an additional property, so that same string do not need to register twice. He tries to design two class `Registry` and `RString`. Please complete his design. (7pts)

```
class Registry:

    def __init__(self, init=[]):
        self.registry = init

    def register(self, string):
        if not string in self.registry:
            self.registry.append(string)

    def safe(self, string):
        return string in self.registry (1分)

class RString(String):
    registry = Registry()

    def __init__(self, value):
        super().__init__(value) (2分)
        if RString.registry.safe(self.value) (2分) :
            self.registered=True

    def register(self):
        RString.registry.register(self.value) (1分)
        super().register() (1分)
```

What would python display? (4pts)

```
>>> b, c = RString("love"), RString("SICP")
>>> b.registry = Registry(["love", "SICP"])
>>> print(b, c)
Not registered string Not registered string (2分)
>>> b.register()
>>> print(b, c)
love Not registered string (2分)
```

Tom want to make it more convenient, like `__add__`. What would python display now? (2pts)

```
>>> b, c, o = RString("love"), RString("SICP"), RString("␣")
>>> b.register(), c.register(), o.register()
>>> print(b, c)
love SICP （1分）
>>> print(b + o + c)
Not registered string （1分）
```

So he adds the following code, (4pts)

```
class RString(RString （1分） ):

    def __add__(self, other):
        string = self.value + other.value
        if self.registered and other.registered:
            RString.registry.register(string) （2分）
        return RString(string) （1分）
```

Now everything should work just fine.

```
>>> b, c, o = RString("love"), RString("SICP"), RString("␣")
>>> b.register(), c.register(), o.register()
>>> print(b + o + c)
love SICP
```

2 Linked Lists (12pts)

2.1 Split (7pts)

Fill in the blanks below to implement `split`. The `split` operator takes a `link` and a predicate `cond` that takes in element's value and evaluates to `bool`, and returns a tuple of two `Link`s. The first returned `Link` contains only the elements of the original `Link` on which `cond` evaluates to `True`. The second returned `Link` contains only the `elements` of the original `Link` on which `cond` evaluates to `False`. The relative order of the elements in the returned `Link`s should be the same as in the original `Link`. If the input linked list is empty, `split` returns two empty linked lists.

```
def split(link, cond):
    """
    >>> is_even = lambda x: x % 2 == 0
    >>> evens, odds = split(Link(1, Link(2, Link(3, Link(4)))), is_even)
    >>> print(evens)
    <2 4>
    >>> print(odds)
    <1 3>
    """
```

```
    if link is Link.empty:
        return Link.empty, Link.empty
    else:
        t1, t2 = split(link.rest, cond) (2分)
        if cond(link.first) (1分) :
            return Link(link.first, t1), t2 (2分)
        else:
            return t1, Link(link.first, t2) (2分)
```

2.2 Iter (2pts)

The `iter` operator returns a new iterator that iterates though the linked list.

```
def iter(link):
    """
    >>> link = Link(1, Link(2, Link(3)))
    >>> for item in iter(link):
    ...     print(item)
    1
    2
    3
    """
    while link is not Link.empty:
        yield link.first (1分)
        link = link.rest (1分)
```

2.3 Convert From List (3pts)

The `convertFromList` function takes a python list and converts it to a linked list.

```
def convertFromList(lst):
"""
>>> link = convertFromList([1, 2, 3])
>>> print(link)
<1 2 3>
"""
if lst == []:
    return Link.empty (1分)
else:
    return Link(lst[0], convertFromList(lst[1:])) (2分)
```

3 Scheme (14pts)

In this problem, you can use any cs61a scheme builtin function, including `append`. But note that `reverse` is *not* a builtin function in cs61a scheme.

The `pair?` functions is used to test if the input is a cons constructed value. E.g.

```
scm> (pair? '())
#f
scm> (pair? 1)
#f
scm> (pair? '(1 2))
#t
scm> (pair? (cons 1 nil))
#t
```

3.1 Snoc (3pts)

We all love the magic function `cons`, which constructs a list by a value and a list. There's a reversed version of `cons`, called `snoc`, which constructs a list by a list and a value. `(snoc l v)` will insert `v` to the end of `l`.

```
scm> (snoc (snoc (snoc '() 1) 2) 3)
(1 2 3)
```

Complete the definition of `snoc`:

```
; scm> (snoc (snoc (snoc '() 1) 2) 3)
; (1 2 3)
(define (snoc l x)
  (append l (list x)) (3分) )
```

3.2 Full Concat (5pts)

`concat` is commonly-used function in functional programming. `concat` will concat a list of list to a list. E.g.

```
scm> (concat '((1 2 3) (4 5)))
(1 2 3 4 5)
```

Besides the normal `concat`, we can defined a full, or generalized concat, called `full-concat` in Scheme. This function will flatten a nested list to a list.

- If the input is not a list, this function will wrap it to a list
- If the input is a nested list, this function will turn it to a list.

Complete the definition of `full-concat`. Hint: See Problem 5.3 for the definition of the normal `concat`.

```
; scm> (full-concat '((1 2 3) (4 5 6)))
; (1 2 3 4 5 6)
; scm> (full-concat '((1 2 3) (4 5 6) (7 8 9)))
; (1 2 3 4 5 6 7 8 9)
; scm> (full-concat '((((1 2 3))) 3 (4 5 6) (7 8 9)))
; (1 2 3 3 4 5 6 7 8 9)
;
(define (full-concat l)
  (cond ((null? l) '() (1分) )
        ((pair? l) (append (full-concat (car l)) (full-concat (cdr l))) (2分) )
        (else (list l) (2分) )))
```

3.3 Tail Recursion (4pts)

The above procedure is not tail-recursive.
Fill in the blanks below to complete a tail-recursive implementation. Hint: maybe complete 3.4 first is helpful.

```
(define (full-concat-tail l)
  (define (helper l acc)
    (cond ((null? l) acc)
          ((pair? l)
           (let ((hd (car l)) (tl (cdr l)))
             (cond ((null? hd) (helper tl acc) (1分) )
                   ((pair? hd)
                    (helper (cons (car hd) (cons (cdr hd) tl)) acc) (2分) )
                   (else (helper tl (snoc acc hd)) (1分) ))))
          (else (snoc acc l))))
  (helper l '()))
```

3.4 Explain (2pts)

Explain how `full-concat-tail` convert a nested list to a list.

```
   (full-concat-tail '((1) 3))
=  (helper '((1) 3) '())
=  (helper '(1 () 3) '())
=  (helper '(() 3) '(1)) (2分)
=  (helper '(3) '(1))
=  (helper '() '(1 3))
=  '(1 3)
```

4 Interpreter (15pts)

The evaluator of a Scheme interpreter consists of two mutually-recursive components:

- `eval(expr, env)` that evaluates `expr` in the form of scheme list under the environment `env`.
- `apply(proc, args, env)` that applies `args` in the form of scheme list to the procedure `proc` under the environment `env`.

Given an expression, we can draw the call sequence of `eval()` and `apply()` on evaluating the expression. For example, provided evaluated `(define (f x) (+ x 1))`, the call sequence on evaluating `(* (f 3) 2)` is:

```
eval(Pair('*', Pair(Pair('f', Pair(3, nil)), Pair(2, nil))))
    eval('*')
    eval(Pair('f', Pair(3, nil)))
        eval('f')
        eval(3)
        apply(LambdaProc<f>, [3])
            eval(Pair('+', Pair('x', Pair(1, nil))))
                eval('+')
                eval('x')
                eval(1)
                apply(BuiltinProc<+>, [3, 1])
    eval(2)
    apply(BuiltinProc<*>, [4, 2])
```

Note that the `env` argument of `eval()` and `apply()` is ignored here because we don't focus on it. In addition, the `proc` argument of `apply()` should be in the form of `BuiltinProc<name>` for a built-in procedure and `LambdaProc<name>` for a user-defined procedure.

Here's your task.
Alice defines two symbols (`g` and `h`) in the interpreter. Then she evaluates an expression using these two symbols, getting the result `2024`. With a few pieces of its evaluation's call sequence, could you recover the definitions of symbols and the expression, and fill in the blanks in the call sequence? Note that each call should be fit in each line.

```
scm> (define (g x) (* x 10)) (2分)
g
scm> (define (h x) 203 (1分) )
h
scm> (- (g (h 5)) 6) (2分)
2024
```

```
eval(Pair('-', Pair(Pair('g', Pair(Pair('h', Pair(5, nil)), nil)) (2分) , Pair(6, nil))))
    eval('-')
    eval(Pair('g', Pair(Pair('h', Pair(5, nil)), nil) (1分) ))
        eval('g')
        eval(Pair('h', Pair(5, nil))) (2分)
            eval('h')
            eval(5)
            apply(LambdaProc<h>, [5])
                eval(203)
        apply(LambdaProc<g> (1分) , [203])
            eval(Pair('*', Pair('x', Pair(10, nil)) (2分) ))
                eval('*')
                eval('x')
                eval(10)
                apply(BuiltinProc<*>, [203, 10] (2分) )
    eval(6)
    apply(BuiltinProc<->, [2030, 6])
```

5 Macro (8pts)

Hint: In Scheme, `'x` is short for `(quote x)`, `` `x `` is short for `(quasiquote x)`, and `,x` is short for `(unquote x)`.

5.1 List comprehension in scheme (2pts)

In the programming language Racket (a dialect of Scheme), there're some macros for list comprehension, such as `for/list` and `for/list*`.

The macro `(for/list binding body cond)` enables a for loop-like functionality for iterating over lists. This iterating results in a new list. It takes three parameters:

- `binding`: A list containing the iteration variable and the list to iterate over.
- `body`: The expression to be evaluated on each iteration.
- `cond`: The condition determining whether to include the result in the final list.

For example,

```
scm> (define odd (lambda (n) (if (= n 0) #f (even (- n 1)))))
odd
scm> (define even (lambda (n) (if (= n 0) #t (odd (- n 1)))))
even
scm> (for/list (i (list 1 2 3 4 5 6 7 8 9 10))
               (* i i)
               (odd i))
(1 9 25 49 81)
```

Now, consider such Python list comprehension, can you give an equivalence scheme list comprehension expression with `for/list`?

```
numbers = [1, 2, 3, 4, 5]
result = [x * 2 for x in numbers if x != 3]
```

```
scm> (define numbers `(1 2 3 4 5))
scm> (define result
       (for/list (x numbers) (* x 2) (not (= x 3)) (1分) ))
```

The macro `(for/list* bindings body cond)` is similar to the JOIN operation in SQL, allowing the combination of multiple iterations or lists based on certain conditions.

This macro enables nested iterations by taking multiple bindings, akin to multiple tables in a SQL database, and generates a new list by combining the elements from these bindings. It filters elements based on the given condition, somewhat resembling the WHERE clause in SQL.

The resulting list comprises combinations of elements from these lists that satisfy the given condition, resembling the output of a SQL JOIN operation.

For example,

```
scm> (for/list* ((i (list 1 2 3))
                 (j (list 2 3 4)))
                (list i j)
                #t)
scm> ((1 2) (1 3) (1 4) (2 2) (2 3) (2 4) (3 2) (3 3) (3 4))
```

Now, filling the blanks for the output of `for/list*`

```
scm> (for/list* ((i (list 1 2 3))
                 (j (list 2 3 4))
                 (k (list 3 4 5)))
                (* i j k)
                (and (>= i j) (>= j k)))
scm> (27) (1分)
```

5.2 Implements for/list (4pts)

Implement a Scheme macro `for/list` described before.

NOTE: `letrec` is similar to `let`, excepts the recursive definition is allowed in `letrec`. E.g.

```
scm> (letrec ((fact (lambda (n) (if (= n 0) 1 (* n (fact (- n 1)))))))
       (fact 10)) ; => 3628800
scm> (let ((fact (lambda (n) (if (= n 0) 1 (* n (fact (- n 1)))))))
       (fact 10)) ; error, fact has no binding
```

In your definition, you *may* use the `caar`, `cadr`, `cdar`, `cadar` defined below.

```
scm> (define (caar x) (car (car x)))
scm> (define (cadr x) (car (cdr x)))
scm> (define (cdar x) (cdr (car x)))
scm> (define (cadar x) (car (cdr (car x))))
scm> (define-macro (for/list binding body cond)
        ;; the definition is just a letrec expression
        ;; here we define a recursive function, named `loop`, like in `fact`
        ;; the body of loop is a lambda expression, with a parameter named `t`
        `(letrec ((loop (lambda (t)
                    (if (null? t)
                        '() (1分)
                        (let ((,(car binding) (car t)))
                            (if ,cond (1分)
                                (cons ,body (loop (cdr t))) (1分)
                                (loop (cdr t))))))))
           ;; the next line is the `expr` part of `(letrec bindings expr)`
           (loop ,(cadr binding)) (1分) )))
```

5.3 Implements for/list* (2pts)

Implement a Scheme macro `for/list` described before. It's a little complicated, we use a helper function `concat` and a helper macro `for/list**`. Your task is to complete `for/list**`.

```
scm> (define (concat l)
       (if (null? l) `()
           (append (car l) (concat (cdr l)))))

scm> (define-macro (for/list** bindings body cond)
       (cond
         ((= (length bindings) 1)
          `(list (for/list ,(car bindings) ,body ,cond))) (1分) )
         (else `(for/list ,(car bindings)
                  (concat (for/list** ,(cdr bindings) ,body ,cond))
                  #t (1分) ))))

scm> (define-macro (for/list* bindings body cond)
       (cond ((null? bindings) `()) ; ill-formed, ignore
             (else `(concat (for/list** ,bindings ,body ,cond)))))
```

6 Streams (11pts)

First, you are given a stream of natural numbers, starting from `start`. The code is already given.

```
; 0, 1, 2, 3, ...
(define (naturals start) (cons-stream start (naturals (+ start 1))))
```

Then, fill in the blanks below to implement `combine-with`, which is a procedure that calculates a stream containing the results of two corresponding elements into `f` function . (4pts)

```
; scm> (define evens (combine-with + (naturals 0) (naturals 0)))
; evens
; scm> (slice evens 0 10)
; (0 2 4 6 8 10 12 14 16 18)
(define (combine-with f xs ys)
  (if (or (null? xs) (null? ys))
      nil
      (cons-stream （1分）
          (f (car xs) (car ys) （1分） )
          (combine-with f (cdr-stream xs) (cdr-stream ys) （2分） ))))
```

In addition, fill in the blanks below to implement `factorials`, which is a procedure that calculates a stream containing a sequence of **factorial** numbers, start from the factorial of 0. (2pts)

```
; scm> (slice factorials 0 10)
; (1 1 2 6 24 120 720 5040 40320 362880)
(define factorials
  (cons-stream 1 (combine-with * (naturals 1) factorials) （2分） ))
```

Next, fill in the blanks below to implement `double-factorials`, which is a procedure that calculates a stream containing a sequence of **double factorial** numbers, start from the factorial of 0. (2pts)

For example, the double factorials of 7 is $1 \times 3 \times 5 \times 7 = 105$.

```
; scm> (slice double-factorials 0 10)
; (1 1 2 3 8 15 48 105 384 945)
(define double-factorials
  (cons-stream 1 (cons-stream 1 (combine-with * (naturals 2) double-factorials) （2分） ))
```

Finally, implement `sub-factorial-stream` (3pts)

It is a procedure that calculates a stream containing the difference between every pair of adjacent（相邻的）factorial elements. It will be like the belowing form, where ! means factorial.

$$(0!,\ \ 1! - 0!,\ \ 2! - 1!,\ \ 3! - 2!,\ \ 4! - 3!,\ \ \cdots)$$

```
; scm> (slice sub-factorial-stream 0 10)
; (1 0 1 4 18 96 600 4320 35280 322560)
(define sub-factorial-stream
  (cons-stream 1 (combine-with - (cdr-stream factorials) factorials)) （3分） )
```

7 SQL (15pts)

In this problem, we will deal with health data of students. You do **not** need to consider calculation's precision .

As background, Body Mass Index (BMI) is a measure that uses your *height* (meters) and *weight* (kilograms) to work out if your weight is healthy. We calculate it with the below formula:

$$\text{BMI} = \frac{weight}{height^2}$$

We have a table `bmi_range` defining BMI range:

```
CREATE TABLE bmi_range AS
    SELECT "Underweight" AS category, 0 as low,  18.5 AS high UNION
    SELECT "Normal range",            18.5,      23.0         UNION
    SELECT "Overweight I",            23.0,      25.0         UNION
    SELECT "Overweight II",           25.0,      30           UNION
    SELECT "Overweight III",          30.0,      100.0;
```

Each row of table `bmi` defines half-open interval of each BMI category. For example, a student with BMI of *22.9* is normal, with BMI of *23.0* is overweight.

And any person at around 20 years old should sleep averagely *7* to *9* hours (allows equal) per day. What's more, he/she should have a regular schedule: never sleep less than 5 hours a day.

There are another two tables, students and sleep.

- The `students` table records `name`, `gender`, `weight` and `height` of each student.
- The `sleep` table records several sleep duration (hours) of Alice and David at some days (you answer should not contain any specific student name or date).

```
CREATE TABLE students AS
    SELECT "Alice" AS name, "Female" as gender , 60 AS weight, 1.70 AS height UNION
    SELECT "Ben"          , "Male"             , 65,          1.80           UNION
    SELECT "Chloe"        , "Female"           , 55,          1.60           UNION
    SELECT "David"        , "Male"             , 75,          1.75           UNION
    SELECT "Emma"         , "Female"           , 45,          1.65           UNION
    SELECT "Finn"         , "Male"             , 50,          1.65           ;

CREATE TABLE sleep AS
    SELECT "2023-12-24" AS date, "Alice" AS name,   8.0  as duration UNION
    SELECT "2024-12-31"        , "Alice",           7.5  as duration UNION
    SELECT "2024-01-04"        , "Alice",           8.0  as duration UNION
    SELECT "2024-01-05"        , "Alice",           8.5  as duration UNION
    SELECT "2023-12-24"        , "David",           8.0  as duration UNION
    SELECT "2023-12-31"        , "David",           6.0  as duration UNION
    SELECT "2024-01-04"        , "David",           4.0  as duration UNION
    SELECT "2024-01-05"        , "David",          12.0  as duration  ;
```

name	gender	bmi	category
Alice	Female	20.8	Normal range
Ben	Male	20.1	Normal range
Chloe	Female	21.5	Normal range
David	Male	24.5	Overweight I
Emma	Female	16.5	Underweight
Finn	Male	18.4	Underweight

Expected results for problem 7.1

name	average
Alice	8.0

Expected results for problem 7.2

summary
Alice has Normal range BMI, and has average sleep of 8.0 hours.

Expected results for problem 7.3

7.1 BMI calculation (6pts)

Write an SQL statement that calculates BMI for every students. The final table is named as `stu_bmi`.

Your result should contain four columns `name`, `gender`, `bmi` and `category`, as shown in the above table.

```
CREATE TABLE stu_bmi AS
    SELECT name, gender, weight / (height * height) AS bmi, category (0.5, 0.5, 1.5, 0.5) (3分)
    FROM students, bmi_range (1分)
    WHERE low <= bmi AND bmi < high (2分)
    ;
SELECT * FROM stu_bmi;
```

7.2 Sleeping well? (6pts)

Write an SQL statement that collects all students with healthy sleeping habit (appropriate sleeping time and regular sleeping routine). The final table is named as `stu_sleep_well`.

Your result should contain two columns: `name` and `average` as shown in the above table.

```
CREATE TABLE stu_sleep_well AS
    SELECT name, AVG(duration) AS average (2分)
    FROM sleep (0.5分)
    GROUP BY name (0.5分)
    HAVING 7 <= average and average <= 9 and MIN(duration) >= 5.0 (1, 1, 1) (3分)
    ;
SELECT * FROM stu_sleep_well;
```

7.3 Health summary generation (3pts)

Write an SQL statement to generate a (one-line) string summary for students appeared in table `stu_sleep_well`.

Your result should contain one columns: `summary`, as shown in the above table.

```
CREATE TABLE stu_summary AS
    SELECT stu_bmi.name || " has " || category || " BMI, and has average sleep of "  (2分)
    || average || " hours." AS summary (错一处-0.5)
    FROM stu_bmi, stu_sleep_well
    WHERE stu_bmi.name = stu_sleep_well.name (1分)
    ;
SELECT * FROM stu_summary;
```